Operating Systems Project

Interactive Memory Allocation Simulator

Project Documentation

Prepared by

Umang Vekariya

Sarthak Dhawale

Aditya Khunt

14 October 2026

Report purpose

This report explains the design, implementation, and evaluation of our browser-based simulator for contiguous memory allocation. It covers First Fit, Best Fit, Worst Fit, and Next Fit, with fixed and variable partitioning. It also provides a user guide and material for a project demonstration or viva.

Abstract

The simulator makes memory placement decisions visible. Users configure regions, create processes, allocate or release memory, request growth, and inspect fragmentation. A shared JavaScript engine performs the state transitions while the interface displays proportional maps, search trails, lifecycle records, and computed metrics. Next Fit stores a resume address and searches circularly. Independent workload replay allows all four strategies to receive the same initial memory and sequence of actions.

A worked example shows a 400 MB request failing despite 550 MB of total free memory because the largest hole is 250 MB. Compaction creates a 550 MB contiguous region, after which the retry succeeds. A second workload demonstrates that Best Fit allocates four requests while the other strategies allocate three. These outcomes describe particular examples rather than a universal ranking of algorithms.

The current implementation passes 48 regression checks and a deterministic stress sequence of 4,000 mixed operations. The project models allocation decisions and simulated MB address offsets. It does not allocate real system RAM or perform kernel operations.

Contents

Report purpose and abstract....1
1 Project overview....3
2 Requirements and capabilities....4
3 Architecture and folder structure....5
4 Data model and partition modes....6
5 Placement algorithms....7
6 Next Fit search behavior....8
7 Process lifecycle and invariants....9
8 Metrics and failure diagnostics....10
9 Worked fragmentation example....11
10 Algorithm comparison and results....12
11 User guide....13
12 Persistence and input validation....14
13 Testing and verification....15
14 Limitations and conclusion....16
Appendix A Viva preparation....17
References and glossary....18

Reading guide

Read Chapters 1 to 4 for the project purpose and architecture. Chapters 5 to 10 explain the allocation model and its results. Chapters 11 to 13 support running, demonstrating, and verifying the application. The final sections describe its limits and prepare the team for technical questions.

All sizes use whole MB values. Address ranges are simulated offsets with inclusive endpoints. Code paths in this report are relative to the project root, which contains index.html.

1 Project overview

Problem statement

A memory request may fit several free regions, only one region, or none. Students need to understand how a placement rule changes the layout and why a later request can fail. A static algorithm description does not show the effects of splitting, release, merging, or repeated requests. Our project provides a local experiment in which users can observe those effects and explain them using the current memory state.

Project objectives

We aim to demonstrate contiguous allocation through four strategies, expose their search decisions, and distinguish fixed reservation from variable splitting. The simulator records successful and failed attempts, computes fragmentation and utilization, and demonstrates recovery through compaction. Its comparison feature replays identical actions on separate states so that differences arise from placement behavior rather than different starting conditions.

The interface also supports a short Overview example, waiting-process previews, a guided fragmentation lesson, and printable reports. The visual Next Fit pointer shows where its next search starts. Explanations beside the metrics use the current values, allowing a presenter to connect a formula to an observed result.

Project scope

The model represents memory as an ordered list of contiguous regions. Each allocated process owns exactly one region. Users control request sizes, partition mode, and placement strategy. Variable-mode growth consumes space immediately after an allocation. Fixed-mode growth uses capacity inside the existing partition. The model records actions as simulation steps rather than elapsed CPU time.

Technology choice

HTML defines the pages, CSS controls presentation, and plain JavaScript performs the simulation. Python provides an optional static file server. Node.js runs the regression suite. The application has no backend service, database, account system, package installation, or build requirement. Browser-local storage can preserve committed workload actions between page loads.

The project is suitable for teaching and demonstration because the core allocation logic remains separate from the interface. It is a simplified model of selected OS concepts rather than a real allocator for operating-system processes.

2 Requirements and capabilities

Functional requirements

Capability	Required behavior
Memory configuration	Accept positive whole-MB region sizes and select fixed or variable mode.
Process management	Create unique processes and track waiting, allocated, and terminated states.
Placement	Offer First Fit, Best Fit, Worst Fit, and circular Next Fit.
Lifecycle operations	Allocate, release, grow in place, merge eligible free holes, and compact variable memory.
Inspection	Display addresses, reserved and requested sizes, search trails, and failure diagnostics.
Comparison	Replay the same workload independently and show four layouts with exact metrics.
Portable experiments	Save and load validated JSON workloads and export a standalone HTML report.

Usability and reliability requirements

The application provides keyboard-accessible navigation and block inspection, light and dark themes, and responsive page layouts. Step mode must commit an allocation exactly once even when users review earlier explanation steps. Previews must not record attempts or mutate memory. Configuration changes must invalidate unfinished previews and clear results that depend on the previous workload.

Memory capacity must remain constant during allocation, release, growth, merging, and compaction. Zero-memory or fully allocated states must produce finite metrics. Workload imports must validate their schema and replay into a separate state before replacing the live session. Reports and interface text must escape user-provided values.

Environment

A modern browser is sufficient to run the application. Python 3 is useful for serving the static files at a localhost address. Node.js is required only for the automated checks. No network connection is needed for the simulator assets after the project files are available locally.

3 Architecture and folder structure

The application separates the memory model from the code that receives user input and renders the result. The live simulation object is the source of truth. Interface components read that state rather than deciding allocation independently.

Layer	Location	Responsibility
Core	js/core/	Placement algorithms, state transitions, address calculations, and statistics.
Controller	js/app.js	Startup, event handlers, comparison runs, step mode, and coordinated rendering.
Interface	js/interface/	Maps, tables, navigation, charts, scenarios, previews, playback, persistence, and reports.
Structure and styling	index.html and css/	Page controls and responsive visual presentation.
Verification and guides	tests/ and docs/	Regression checks, preview pages, explanation guides, and presentation material.

Application flow

When the user chooses Allocate, the controller locates the waiting process and invalidates any stale explanation preview. The engine requests a pure search result from the selected algorithm. On success, it reserves or splits the region, updates process ownership, recalculates addresses, records the event, and updates metrics. The controller then refreshes the interface from the new state.

```
User action
  app.js handles the request
  MemoryEngine calls a placement function
  algorithms.js chooses a suitable region
  MemoryEngine updates the state
  renderApplication refreshes the interface
```

Entry points

The browser opens index.html. Its script list loads algorithms and the engine before the interface helpers, then loads app.js last. Scripts share the page scope; the folder layout does not imply independent ES modules. The controller initializes a memory configuration, restores a valid saved session if available, attaches handlers, and activates the selected workspace page.

README.md provides setup instructions. PROJECT_GUIDE.md explains each file. PRESENTATION.md supplies a speaking order, while TECHNICAL_REFERENCE.md records detailed assumptions and formulas. The separate presentation file contains editable charts and speaker notes.

4 Data model and partition modes

Simulation state

MemoryEngine.state creates the live model. It contains configuration, memory regions, processes, workload actions, metrics, events, chart samples, Next Fit position, and the latest diagnostic or compaction snapshot. Each independent comparison creates its own state.

Record	Important fields
Memory region	id, size, startAddress, endAddress, free, processId, processName
Process	id, name, requestedMemory, allocatedMemory, status, blockId, allocationAttempts, allocationFailures
Configuration	totalMemory, mode, algorithm, initialMemory, initialAlgorithm, nextFitAddress
History	workload, events, analytics, step, lastFailure, lastCompaction
Presentation state	comparisonResults, scenario, scenarioCursor, stepSimulation, guide

Address representation

Addresses are simulated MB offsets. A region starting at 0 with size 200 ends at 199. The following region starts at 200. The engine recalculates ranges in list order after layout changes, ensuring that the regions cover the configured space without overlaps or gaps in the address representation.

Variable partitions

A successful request consumes exactly its required size. The selected free region becomes allocated and any remainder becomes a new free region. Release and growth can coalesce adjacent holes. Compaction moves allocated regions toward address zero and combines all free space. Configured variable regions initially remain independently selectable for textbook hole-list examples, even if their simulated ranges are adjacent.

Fixed partitions

A successful request reserves the entire selected partition. The difference between reserved and requested size is internal waste. Partition sizes and boundaries remain fixed. Free partitions do not merge and compaction is disabled. A process may grow only within the unused capacity of its already reserved partition.

5 Placement algorithms

The algorithms return a selected block, a trail of examined free blocks, and the leftover size. They do not modify memory. The engine performs validation and state changes. First Fit, Best Fit, and Worst Fit use the ordered region list, while Next Fit also receives a saved resume address.

Strategy	Selection	Search behavior
First Fit	First suitable free region	Stop at the first fit from the beginning.
Best Fit	Smallest suitable free region	Scan all free regions.
Worst Fit	Largest suitable free region	Scan all free regions.
Next Fit	First fit from the saved position	Scan circularly and stop at a fit.

First Fit

The search begins at the first region, skips occupied regions, and returns as soon as a free region can satisfy the request. Its trail ends at the selected block. This can reduce examinations for requests that fit early in the list.

Best Fit and Worst Fit

Best Fit retains the smallest suitable region found during the scan. Worst Fit retains the largest. Both examine every free region in the current implementation. Equal-sized candidates retain the earliest candidate because the code replaces a selection only when it finds a strictly smaller or strictly larger suitable region.

Algorithm result

```
success: whether a suitable region exists
selectedBlockId: chosen region or null
checkedBlocks: ordered free-region search trail
suitableBlocks: suitable candidates found
leftover: chosen region size minus request size
```

Complexity and interpretation

All four strategies have O(n) worst-case search time for n regions in this list-based implementation. The recorded search metric counts free regions examined rather than elapsed execution time. A strategy that searches fewer regions need not serve more requests. The workload and release pattern determine the observed outcome.

6 Next Fit search behavior

Next Fit is a circular variation of First Fit. The simulator stores nextFitAddress as a simulated MB address. Each search starts in the region containing that address, skips allocated regions, and wraps to the beginning when necessary. It visits each region at most once, so an unsuccessful search terminates after a single complete cycle.

Search pseudocode

```
cursor = saved address modulo total memory
start = region containing cursor
for offset from 0 to number of regions - 1:
    index = (start + offset) modulo region count
    region = memory[index]
    if region is free and size >= request:
        return region and recorded search trail
return allocation failure
```

Cursor updates

Action	Effect on the saved address
Successful Next Fit allocation	End of the reserved region plus one, wrapping to zero.
Failed allocation or read-only preview	No change.
Step review or algorithm switch	No change.
Release, merge, or in-place growth	Retain the address; find its containing region again.
Configuration, reset, or compaction	Reset to zero.
Workload import or comparison	Reconstruct a cursor in that independent replay state.

Why the model stores an address

A list index can become invalid when a split inserts a region or a merge removes one. An address remains meaningful across those operations. If merging places the saved address inside a free hole, this implementation considers the whole containing region. In variable mode the next request can use the leftover hole from the previous allocation. Fixed mode normally resumes at the following partition.

The visible pointer marks the search starting position, not a guarantee that the next allocation will use that region. Allocated or undersized regions may be skipped, and the search may wrap. The mini example on Overview uses a separate illustrative saved position and never changes the live session.

7 Process lifecycle and invariants

Creation and allocation

A process begins in the waiting state after successful validation. The engine requires a nonempty ID and name, a positive safe whole-MB request, a unique ID, and configured memory. Allocation searches for a region. Success sets ownership, requested and reserved sizes, the allocation step, and the process state. Failure keeps the process waiting and records the attempt, reason, and diagnostic metrics.

Release and merging

Termination applies to allocated processes. It releases the owned region, clears ownership, and records the terminated state. In variable mode the engine merges adjacent free regions; fixed mode preserves partition boundaries. A released process retains its lifecycle record rather than becoming a new waiting request.

Additional memory requests

Growth changes an existing allocation in place. Variable mode uses the following contiguous free region after eligible coalescing. It never moves the process to a different region or duplicates it. Fixed mode uses unused capacity inside the reserved partition. A failed growth request preserves the process allocation and requested size, though coalescing performed before the check can simplify the surrounding free layout.

Compaction

Compaction collects allocated regions in their existing order, moves them toward address zero, and appends one free region. It records before and after layouts, preserved free capacity, and process movements. Its result is a snapshot: later allocations appear in the live layout without altering those recorded compaction maps.

State invariants

The sum of region sizes equals configured total memory. Every region has a positive whole-MB size and unique block ID. Address ranges are contiguous and ordered. Each allocated process owns exactly one region, and its reserved size is at least its requested size. Free plus reserved memory equals total memory. Internal and external fragmentation remain nonnegative.

Step mode previews the same engine search and commits the resulting action exactly once. Previous reviews the explanation rather than undoing memory. Relevant configuration or lifecycle changes cancel an unfinished preview so it cannot commit an outdated decision.

8 Metrics and failure diagnostics

Metric	Definition
Reserved utilization	Reserved allocated memory / total memory x 100.
Useful utilization	Requested memory of allocated processes / total memory x 100.
Internal fragmentation	Sum of reserved size minus requested size in allocated processes.
External fragmentation	Free memory minus the largest free hole in variable mode.
External ratio	External fragmentation / free memory x 100.
Internal ratio	Internal fragmentation / reserved memory x 100.
Search effort	Free regions examined by valid allocation and growth attempts.

Interpreting waste

For a 200 MB process in a 300 MB fixed partition, reserved memory is 300 MB and useful payload is 200 MB. Internal waste is 100 MB. In an 800 MB configuration, reserved utilization is 37.5% and useful utilization is 25%. Variable allocations reserve their requested size, so this model reports zero internal waste for them.

External fragmentation describes dispersed free memory. It remains available for smaller requests. Fixed mode reports its external metric as zero and diagnoses unusable free partitions as partition-capacity limits. The interface total-waste value adds internal waste and the variable external metric; it is not a measure of permanently lost bytes. Ratios use zero when their denominator is zero.

Failure classification

The engine distinguishes insufficient total free memory, fragmented variable space, fixed partition limits, and inadequate adjacent space for growth. A diagnostic records the request size, total free memory, largest region, number of free regions, process ID, and simulation step. These values describe the failed attempt rather than continuously changing with later actions.

Recovery suggestions follow the cause. Compaction can help when enough variable free space exists in separate holes. It cannot create capacity when total free memory is insufficient. Fixed mode needs an adequate partition. Growth may require freeing adjacent memory or retrying after an eligible layout change.

9 Worked fragmentation example

The guided example starts with 1,000 MB in variable mode. Processes P1 to P5 request 250, 200, 250, 250, and 50 MB and fill the space. Releasing P1, P3, and P5 leaves P2 and P4 allocated. The resulting holes are 250, 250, and 50 MB.

Region	Before compaction	Size MB
Free	0 to 249	250
P2	250 to 449	200
Free	450 to 699	250
P4	700 to 949	250
Free	950 to 999	50

Why P6 fails

P6 requests 400 MB. Total free space is 550 MB, but the largest hole is only 250 MB. No strategy can place this contiguous request in the current layout. The external-fragmentation metric is 550 minus 250, or 300 MB. The failed attempt remains part of P6’s history.

Compaction and retry

Region	After compaction	After allocating P6
P2	0 to 199	0 to 199
P4	200 to 449	200 to 449
P6	Waiting	450 to 849
Free	450 to 999	850 to 999

Compaction moves P2 by 250 MB toward zero and P4 by 500 MB. Total free memory remains 550 MB, while the largest hole becomes 550 MB. The retry succeeds and reserves 400 MB for P6. Final allocated memory is 850 MB, free memory is 150 MB, and reserved utilization is 85%.

This sequence separates three ideas: releasing memory creates holes, merging joins holes that are already adjacent, and compaction relocates allocated processes so separated holes can become one. It provides a repeatable live demonstration of failure and recovery.

10 Algorithm comparison and results

The comparison workload starts with regions of 100, 500, 200, 300, and 600 MB, totaling 1,700 MB. It attempts requests of 212, 417, 112, and 426 MB in that order. Each algorithm receives a separate state with identical starting memory and actions.

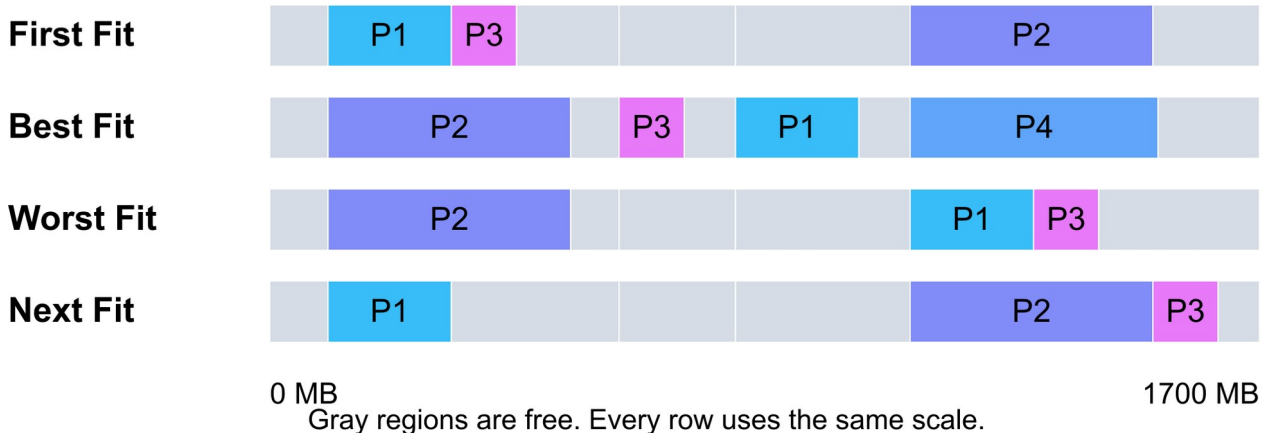


Figure 1 Final variable-memory layouts computed by the current engine

Algorithm	Placed	Searches	Useful use	External MB
First Fit	3 of 4	14	43.6%	659
Best Fit	4 of 4	20	68.6%	359
Worst Fit	3 of 4	20	43.6%	659
Next Fit	3 of 4	12	43.6%	659

Best Fit serves the additional 426 MB request in this example. Next Fit examines fewer free regions than the other strategies. These measured results demonstrate a workload-dependent tradeoff between successful placement and search effort. They do not establish an algorithm that is always best.

Fair replay and snapshots

Comparison uses the original configuration and recorded lifecycle actions, excludes manual algorithm-change and event-note actions, and can append remaining selected-scenario actions. It also attempts waiting processes that have never had an allocation action. Invalid lifecycle actions do not count as allocation failures. Comparisons never alter live memory. Later live edits mark their maps stale, and configuration or reset clears them.

Recommendation score

The project-specific score weights success rate at 65%, useful payload utilization at 15%, the complement of external fragmentation ratio at 15%, and normalized search economy at 5%. Search economy is 100 x (1 - searches / maximum searches across algorithms), using at least 1 as the denominator. Exact score ties retain every winning strategy.

11 User guide

Running the application

Open a terminal in the project root and start the optional local server. Python serves the files; the browser executes the simulation logic. If the selected port is occupied, choose another available port and use its matching URL.

```
python3 -m http.server 8001 --bind 127.0.0.1
```

```
Open http://127.0.0.1:8001/ in a browser
```

The app can also run offline by opening index.html directly. A local server is recommended for consistent browser behavior. Stop it with Ctrl+C when finished.

Configuring and allocating

Open Simulation studio. Choose an algorithm and partition mode, enter comma-separated region sizes, and configure memory. Create a process with its ID, name, and required MB. Select the waiting process and allocate it. A waiting-queue chip also selects the request. Request preview compares the current next placement without changing memory.

Inspection and step mode

Click a proportional memory segment to inspect its address range, reserved capacity, useful payload, waste, and available in-place growth. The detailed list provides another way to inspect blocks. Enable step mode before allocation to review the examined regions, selection, split or reservation, and commit. Play, Pause, and speed controls advance the same explanation. Previous reviews an earlier step without reversing the allocation.

Demonstrations and comparison

Start guided demo to follow the fragmentation sequence described in Chapter 9. Advance stages in order and avoid unrelated changes while the lesson runs. Scenario Next Action performs one lifecycle action, while Run Workload executes the remainder or pauses for a step-mode allocation. Loading a scenario replaces the current workload after confirmation when applicable.

On Compare algorithms, run comparison and inspect the four maps, their address tables, and exact metrics. On Memory insights, inspect trends and failures. Activity log records events. Overview provides a short independent example and the live summary. Theme changes affect presentation, while Reset clears the simulation.

12 Persistence and input validation

Browser autosave

Committed configuration and workload actions save locally in the browser. Reload reconstructs the model by replaying those actions through the same engine. This preserves allocation attempts, failures, algorithm changes, compaction, and scenario progress. Unfinished previews and playback reset on reload. Comparison results should be rerun because they are computed snapshots. Reset stores an empty session.

Local storage is specific to the browser and origin. Opening a different port or a file URL can use different storage. The application does not provide an online account, shared database, or automatic team synchronization. Save a JSON workload when an experiment needs a portable backup.

Workload import format

```
format: "imas-workload"
version: 1
mode and initialAlgorithm
memory: initial region sizes
actions: committed lifecycle records
scenario and scenarioCursor
```

Save workload downloads this versioned JSON structure. Load workload checks the file and replays it into a separate state before replacing the live model. The import does not trust saved statistics or layout fields. A replay failure without a valid allocation diagnostic rejects an impossible lifecycle action rather than partially applying it to the live session.

Validation boundary	Limit or behavior
File size	Up to 2 MB.
Initial memory regions	Up to 1,000.
Recorded actions	Up to 5,000.
Identifiers and names	Nonempty IDs up to 100 characters and names up to 300.
Numeric requests	Positive safe whole-MB values; total capacity must be safe.
Actions and configuration	Recognized action types, valid modes and algorithms, valid scenario progress.

Report export

Export report downloads standalone HTML with configuration, computed statistics, addresses, processes, event history, compaction movements, and the latest comparison if available. Open it in a browser and use Print or Save as PDF. The report has no executable JavaScript or external assets. Rendered user text is escaped in the interface and report output.

13 Testing and verification

Run the dependency-free suite from the project root. It loads scripts in the order specified by index.html and exercises the engine and controller through a lightweight document stub. The current suite passes 48 regression checks.

```
node tests/simulator.test.js
```

Coverage area	Expected behavior checked
Placement and search	Correct region selection and circular wraparound without repeated scans.
Partition modes	Variable splitting and merging, fixed reservation, and internal waste.
Lifecycle and failures	Release, growth limits, diagnostics, compaction conservation, and retry.
Step mode and playback	One commit, valid review behavior, pausing, speed changes, and stale-preview cancellation.
Replay and comparison	Independent states, saved cursor reconstruction, validated imports, and autosave restore.
Teaching views	Pointer position, live metric text, consistent comparison maps, stale notices, and reset.
Rendered text	Escaped IDs and names in reports and comparison layouts.
Scenario coverage	Every predefined scenario under all four algorithms and both partition modes.

Deterministic stress sequence

The suite runs 500 mixed operations for each algorithm and mode, producing 4,000 operations across eight combinations. After every action it checks memory conservation, safe positive region sizes, unique IDs, sequential address ranges, ownership consistency, and nonnegative waste. Fixed-mode partition sizes must remain unchanged.

Interpreting verification

These checks support correctness of the implemented model. They do not measure real OS performance or elapsed search speed. The document stub also cannot establish that a browser layout is visually correct. The responsive and desktop preview pages in tests/ support separate visual inspection, and a presenter should rehearse the full demo in the intended browser before assessment.

Useful manual checks include keyboard navigation, dialog cancellation, responsive table scrolling, light and dark themes, visible pointer updates, report downloads, and session restore. New changes should rerun the suite and repeat relevant visual checks.

14 Limitations and conclusion

Model limitations

The simulator uses whole-MB offsets and one contiguous region per process. It does not model byte alignment, allocation metadata overhead, physical RAM, paging, page tables, virtual addresses, swapping, CPU scheduling, or memory-protection hardware. Real operating systems use additional mechanisms and policies beyond this project's simplified model.

Configured variable regions initially remain independent to reproduce textbook hole-list examples. Release and growth coalesce adjacent free regions, while compaction combines all free space. A stored Next Fit address inside a merged hole causes the search to consider that entire containing region. These documented choices matter when interpreting an experiment.

Compaction changes simulated addresses without modeling its runtime cost or real process relocation. Search effort counts free-region examinations and does not provide elapsed timing. The weighted recommendation score is a teaching aid, not a universal performance ranking. Results depend on the memory configuration and the full sequence of requests and releases.

Possible future extensions

Paging and page-replacement lessons could extend the educational scope, with separate models so contiguous allocation remains clear. Other useful additions include configurable alignment, measured simulation-operation counts, custom scenario authoring, and a richer workload timeline. Any timing benchmark would need an explicit method and repeated measurements rather than inferring speed from search counts.

Accounts or cloud synchronization would require a backend and a separate data-management design. They are optional extensions because the present local simulator already handles its allocation calculations in the browser.

Conclusion

Our project connects memory-allocation rules to visible outcomes. Users can explain where a process fits, why a request fails, and how release or compaction changes the available layout. The shared engine supports consistent behavior across manual actions, previews, replay, and algorithm comparison.

The worked experiments show two important effects: scattered free memory can prevent a contiguous request, and different placement strategies can serve different numbers of requests on the same workload. Next Fit adds the effect of a persistent search position. The diagrams, metrics, and event history make these effects suitable for an OS project demonstration.

Appendix A Viva preparation

Why can allocation fail with enough free memory

The request needs one contiguous region. A sum of separate holes may be large enough while each individual hole is too small.

What distinguishes Next Fit from First Fit

First Fit starts from the beginning. Next Fit resumes from its saved address, wraps around, and stops at the first suitable region.

Is Best Fit always the best strategy

No. The request sizes and release pattern determine the outcome. Best Fit may also create small leftover holes.

How does compaction differ from coalescing

Coalescing merges already adjacent free regions. Compaction moves allocated regions so separated free space can become contiguous.

What is internal fragmentation

Unused memory inside a reserved allocation. A 200 MB process in a 300 MB fixed partition leaves 100 MB of internal waste.

How is comparison fair

Each strategy receives an independent state with the same starting memory, partition mode, and lifecycle actions. Live memory remains unchanged.

Does Previous undo an allocation

No. It reviews the explanation. Completion commits the allocation once, and reviewing a completed sequence does not repeat it.

Where are the algorithms and why is no backend needed

Placement functions live in js/core/algorithms.js. The memory engine performs transitions in JavaScript, so the browser can simulate the entire model locally.

How do we know memory remains consistent

Regression tests and 4,000 mixed operations check capacity conservation, address ranges, region sizes, process ownership, and partition boundaries.

What is outside the current scope

Real RAM allocation, kernel operations, paging, virtual memory, swapping, and CPU scheduling. These are distinct possible extensions.

References and glossary

Project implementation and evidence

1. Interactive Memory Allocation Simulator. Current project implementation, 14 October 2026. Core evidence: js/core/algorithms.js and js/core/memory-engine.js. Controller and comparison: js/app.js. Interactive teaching views: js/interface/learning-views.js. Verification: tests/simulator.test.js.

2. Project guides. README.md, docs/PROJECT_GUIDE.md, docs/TECHNICAL_REFERENCE.md, and docs/PRESENTATION.md. These document the current setup, formulas, assumptions, saved-workload behavior, and demonstration sequence.

Operating Systems background

3. University of Illinois Chicago. Operating Systems course notes, Main Memory, Section 8.3 Contiguous Memory Allocation. Accessed 14 October 2026. https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/8_MainMemory.html

The course notes provide background for contiguous placement, fragmentation, and compaction. The report's numeric results, cursor policies, recommendation score, and verification counts come from the project implementation rather than a claim that all operating systems use the same model.

Glossary

Term	Meaning in this report
Hole	A free memory region that a contiguous request may use.
Payload	Memory requested by processes that currently hold an allocation.
Reserved memory	Capacity assigned to allocated processes, including fixed waste.
Cursor	The saved MB address where a Next Fit search resumes.
Coalescing	Merging adjacent free regions in variable mode.
Compaction	Relocating allocations to combine dispersed free space.
Replay	Reconstructing a state by applying recorded lifecycle actions.
Snapshot	A saved diagnostic, comparison, or compaction result for one point in the experiment.